Computer Security Exam

Professors M. Carminati & S. Zanero

16/07/2021

Last (family) Name _____

First (given) Name _____

Matricola _____

Codice Persona _____

Professor: [ ] Carminati [ ] Zanero

Have you done any challenges/homework, even partially? [ ] Yes [ ] No

Do you want to withdraw ? [ ] Yes [ ] No

Instructions

- **Duration**: 2 hours.
- The exam is composed of 19 pages. Check that you have all of them
- The exam is "open book". The students will be allowed to consult only the **printed course material** and **notes**. **No extra devices** (e.g., phones, iPad) are **allowed**.
- You are not allowed to communicate with other students in any communication form.
- Schemes are good, short answers are recommended.
- Always motivate your answer.
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

- **DISTANCE MODE ONLY**
  - You will be allowed to consult only printed documents or notes as supporting materials. No digital devices (other than the computer you are reading the exam from) will be allowed.
  - On the computer, you will be allowed to open only the MS forms browser tab and the exam pdf. Any other application or document is prohibited.
  - You will have to share your screen, leave your microphone opened, and your webcam must frame both you and your workspace.
  - You are responsible for having all the necessary technological equipment for the correct delivery of the exam.

- If you disable the screen sharing or the audio, you will be expelled from the virtual room and your exam will be voided.
- We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or for a subset, at the instructor's decision.
- Regarding the submission :
  - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
  - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
  - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents
  - We will evaluate only the uploaded **readable** documents
  - USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf)

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 (10 points)

Assume that:

- The C standard library is loaded at a known address during every execution of the program, and that the address of the function `system()` is `0xf4d0e2d3`.
- Environment variables are located in the highest addresses.
- *The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **`cdecl`** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against the exploitation of memory corruption errors is present (unless specified otherwise).*

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <time.h>

typedef struct {
  int *fd;
  char password[8];
  char backdoor[8];
  char key[32];
  char message[32];
  char *p_msg;
} data_t;

void load_secret(data_t *data) {

  data->fd = fopen("secret.txt", "r");
  fscanf(data->fd, "%[^\n]", data->key);
  fclose(data->fd);
  data->key[31]='\0';
  }

void encrypt(data_t *data) {
  int i;

  load_secret(data);

  data->p_msg = data->message;
  scanf("%s", data->message);
  printf("Your message is: %s \n", data->p_msg);
  for (i=0; i<32; ++i){
      data->message[i] = (char)(data->message[i]^data->key[i]);
  }
  printf("Your encrypted message is: ");
  printf(data->message);
}

void main(int argc, char** argv) {
  data_t data;

  data.fd = fopen("password.txt", "r");
  fscanf(data.fd, "%[^\n]", data.password);
  fclose(data.fd);

  scanf("%7s", data.backdoor);
  if (strcmp(data.password, data.backdoor) == 0){
      system("/bin/sh\0");
  } else {
      encrypt(&data);
   }
}
```

[1 point] 1. The program is affected by a typical buffer overflow and format string vulnerability. Complete the following table, focusing on a vulnerability per row.

Vulnerability	**Line**	**Modify the line to solve the detected vulnerability**
Buffer Overflow	30 **[0.25]**	`scanf("%31s", data->message);` **[0.25]**
Format String	36 **[0.25]**	`printf("%s", data->message);` **[0.25]**

From now on assume that the "Stack Canary" mitigation technique is correctly implemented (i.e, it can not be guessed or copied).

[1 point] ***2.*** Draw the stack layout after the program has executed the instruction at line 17, showing:
- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames.

Show also the content of the caller frame.

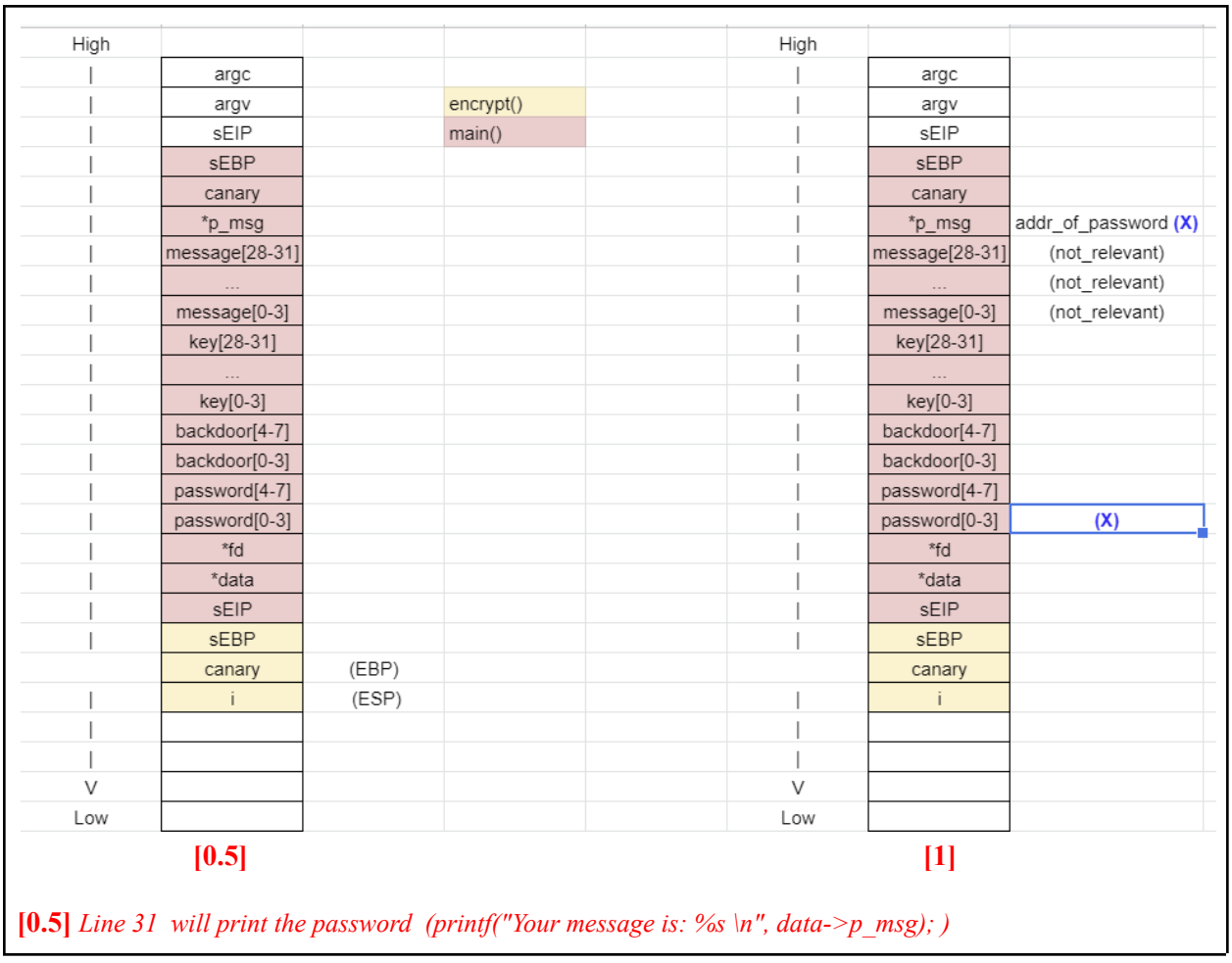
High					
|	argc		load_secret()		strcmp(data.password, data.backdoor) != 0
|	argv		encrypt()		
|	sEIP		main()		
|	sEBP				
	canary				
|	*p_msg				
|	message[28-31]				
|	...				
|	message[0-3]				
|	key[28-31]				
|	...				
|	key[0-3]				
|	backdoor[4-7]				
|	backdoor[0-3]				
|	password[4-7]				
|	password[0-3]				
|	*fd				
|	*data				
|	sEIP				
|	sEBP				
	canary				
|	i				
|	*data				
|	sEIP				
V	sEBP	(ESP)	(EBP)		
Low	canary				

[2 points] 3. <u>Focus only on the stack-based buffer overflow(s) you found.</u>

The attacker wants to leak the value of the variable data.password, which would enable him/her to spawn a privileged shell, as shown in the source code provided.

After the computer security course he/she was able to implement **the following shellcode**, composed of 8 bytes, which leaks the value of the variable data.password: **`0x20 0x30 0x40 0x50 0x60 0x70 0x80 0x90`**.

Is it possible for the attacker to exploit the vulnerability by using the sheelcode he/she has implemented? If yes, write an exploit for this vulnerability that executes the shellcode provided. If not, write an exploit that achieves the same goal (i.e., leaks the value of the variable data.password). Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation. Ensure you describe **all the steps and assumptions** required for a successful exploitation of the vulnerability. Include also any assumption on how you must call the program (e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables, if any).



[0.5] *Line 31 will print the password (printf("Your message is: %s \n", data->p_msg);)*

[2 points] 4. Now consider also the format string vulnerability. Write an exploit that executes the previous shellcode, assuming that you have already prepared the memory with the correct arguments. Assume that:

- The address of the return address (*saved EIP*) of the function exploited is: **`0xe5a0fe4b`** (i.e., where to write)
- The address of the first instruction of the **`shellcode`** is in an **environment variable** at: **`0xcafebabe`** (i.e., what to write)
- The displacement on the stack of the vulnerable function's argument is: **4**

Knowing that *dec(* **`0xcafe`***) = 51966 and dec(***`0xbabe`***)=47806*, write the exploit clearly, detailing all the components of the format string and all the steps that lead to a successful exploitation.

If you need to execute the program multiple times or exploit multiple vulnerabilities, please describe all the steps and assumptions in order to succeed in the exploitation.

[0.5] *Format string vulnerability, in the string data.message, which however is encrypted.*
To exploit the format string it is necessary to leak first the value of key (by exploiting the buffer overflow and printing the value of key at line 31 or by inserting "000000000..."). Then, you build the exploit as usual and provide in input to the vulnerable program (exploit XOR key).

[0.5] *We need to write 0xcafebabe to0xe5a0fe4b. As 0xcafe > 0xbabe, we don't need to swap.*

[0.5] *The general format string structure is: <where to write><where to write + 2>%<low value>c%<pos>$hn%<high value>c%<pos>$hn*
- Where to write = \x4b\xfe\xa0\xe5
- Where to write + 2 = \x4d\xfe\xa0\xe5
- we have already written 8 characters
- low value = 0x49d0 -> 47806 - 8 = 47798
- high value = 0xf7e3 -> 51966 - 47806 = 4160
Also, the displacement on the stack of the printf's argument (i.e., the buffer message) is 4

[0.5] *Complete exploit:* **`\x4b\xfe\xa0\xe5\x4d\xfe\xa0\xe5%47798c$4hn%4160c$5hn`**

[1 point] 5. Consider also the "Address-space layout randomization (ASLR)" mitigation technique (the canary is still active). Is it effective to prevent **your exploits** at point 3. And 4.? If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

[0.5] 3. Yes ,[see slides]*

[0.5] 4. Yes, [see slides]*

*In both cases you need a leak. If the binary is not PIE, you can use ROP or retolibc

[1 point] 6. Consider also the "Non-executable stack (NX)" mitigation technique (the canary is still active). Is it effective to prevent **your exploits** at point 3. and 4? If the mitigation technique is effective,

explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

[0.5] 3. No, [see slides]

[0.5] 4. Yes, [see slides] + ROP or ret_to_libc

[2 points] 7. Consider remote exploitation (i.e., you do not have local access to the machine on which the vulnerable program is executed, not SSH, you can interact only with inputs and outputs). Do the exploits you wrote at point 3. and 4. works without any modification? If the answer is not, explain why and describe how you would modify the exploit. If the answer is yes, explain, please explain why. Assume that no mitigation is enabled.

[1] 3. Yes [see slides], since no mitigation is enabled the exploit will be basically the same. You need to update the addresses

[1] 4. NO, [see slides] the shellcode is in an env variable. Modification: ROP or rettolibc

Exercise 2 (9 points)

You and your friends are about to launch "PayMate", a new online service for exchanging money between acquaintances. You're very excited about this new project. Nonetheless, the simple fact that there is (a lot of) customers' money involved kind of scares the heck out of you, so you want to make sure that everything is on point. In particular, you want to be a hundred percent positive that no one is going to hack your customers and steal their money.

In a nutshell, PayMate allows people to "become friends" within the platform and exchange money with a simple click. Most of the website has been thoroughly pentested, however you and your friends ran out of money before the pentesting process could be completed. As a consequence, some of the web services have not been tested yet. In particular, you can assume that **there are no security issues** with the registration/authentication process and with the management of "friendship" requests. In particular, each user is identified through a username which is **unique, unmodifiable, and that has been checked by a human operator upon registration,** hence it cannot be used as an attacking vector. Moreover, sessions are validated through a token whose creation and management are fail-safe.
On the other hand, as of today, the back-end services that handle the actual transfer of money have not been reviewed or tested for security issues: it's now up to you to roll up your sleeves and do the job.

The PayMate's database contains the following tables:

ACCOUNTS

username (string - primary key)	balance (integer)

FRIENDS

user_1 (foreing key referencing ACCOUNTS.username)	user_2 (foreing key referencing ACCOUNTS.username)

(both fields together constitute the primary key)

MONEY_TRANSFER

id (primary key)	user_1 (foreing key referencing ACCOUNTS.username)	user_2 (foreing key referencing ACCOUNTS.username)	amount (integer)	message (text)

When a user wants to send money to a second user, they need to access the web-service on **https://paymate.com/send_money?[...]**, which is implemented by the following function:

```
01 def send_money(request):
02    if request.method != “POST”:
03          abort(403)
04
05    if not is_session_valid(request):
06          abort(403)
07
08    username = request.args.get(“username”)
09    amount_to_be_sent = request.args.get(“amount”)
10    query = “SELECT balance FROM ACCOUNT WHERE username=” + username + “;”
11    # db_execute returns a list of results, here we take the first and only
12    balance = db_execute_query(query)[0]
13    recipient = request.args.get(“recipient”)
14    if balance > amount_to_be_sent and check_friends(username, recipient):
15          message = request.args.get(“message")
16          pay_recipient(user, recipient, amount, message)
17
18
19 @execute_atomically
20 def pay_recipient(user, recipient, amount, message):
21    query_statement = “INSERT INTO MONEY_TRANSFER ”
22                      “(user_1, user_2, amount, message)”
```

```
23                      "VALUES (%s, %s, %i, %s)"
24    db_execute_query_with_statement(user, recipient, amount, message)
25
26    query_statement = "UPDATE ACCOUNTS SET balance = balance + %i "
                        "WHERE username = %s"
27    db_execute_query_with_statement(-amount, user)
28    db_execute_query_with_statement(amount, recipient)
29
30
31 def check_friends(user_1, user_2):
32    query_statement = "SELECT * FROM FRIENDS WHERE user_1=%s and user_2=%s"
33    r = db_execute_query_with_statement(query_statement, user_1, user_2)
34    return not r.is_empty()  # true if r is not empty
```

In the snippet above, **is_session_valid**, which extracts the username and the authentication token from the request and validates them, can be assumed **to function well and be secure**. Furthermore, **pay_recipient** is always executed atomically and its operations are always locked properly (isolation is guaranteed).

Next, the following two functions are used to collect, respectively, the list of friends for a given user and the list of payments they have received. In particular, the endpoint that is invoked to get the list of friends for a given user is **paymate.com/friends?username=your_username&token=your_auth_token.**

```
35 def get_friend_list(request):  # GET request at paymate.com/friends
36    if not is_session_valid(request) or request.method != "GET":
37          abort(403)
38
39    query_statement = "SELECT user_2 FROM FRIENDS WHERE user_1 = %s"
40    username = request.args.get("username")
41    results = db_execute_query_with_statement(query_statement, username)
42    return json.dumps({"friends": results})  # results is a list/array
43
44
45 def get_received_payments(request):
46    if not is_session_valid(request):
47          abort(403)
48
49    query_statement = "SELECT user_2, amount, message FROM MONEY_TRANSFER "
50                      "WHERE user_1 = %s"
51
52    username = request.args.get("username")
53    results = db_execute_query_with_statement(query_statement, username)
54
55    payments_html = "<table>
56    payments_html += "<tr><th>Sender</th><th>Amount</th>"
57    payments_html += "<th>Message</th></tr>"
58
59    for result in results:
```

```
60          payments_html += "<tr><td>" + result[0] + "</td>"  # sender
61          payments_html += "<tr><td>" + result[1] + "</td>"  # amount
62          payments_html += "<tr><td>" + result[2] + "</td></tr>"  # message
63
64      return payments_html + "</table>"
```

1. [3 points] Fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, be **concise and straight to the point**. If you believe there might be a vulnerability, specify the corresponding line/s of code.

Vulnerability class	*Is there a vulnerability of this class in the code above? How could an adversary exploit it?*	*If the vulnerability is present, explain the simplest procedure to remove it.*
cross-site scripting (XSS) **Specify if reflected, stored, DOM...**	*Yes, there is a stored XSS vulnerability. In particular, a user may insert JS in the message while sending money to a friend. This will be saved in the DB (line 24) and shown to the victim when they query for their received payments (59-62).*	*The simplest procedure to prevent this vulnerability is to apply escaping/filtering to the message.*
Cross site request forgery (CSRF)	*Yes, one could exploit the XSS vulnerability described above to force a user to perform unwanted operations for which authentication is necessary (such as sending money).*	*You simply need to fix the JS vulnerability and put in place checks on the referrer.*
Sql Injection (SQL-I)	*No, there is not any SQL-i vulnerability. At line 12 an SQL query is performed in an unsafe way (i.e., without statements). However, the only user's input is the username, which is validated by the is_session_valid function which is assumed to function well and to be secure.*	.

2. [1 point] Exploit one of the vulnerabilities that you have identified to show a message that says "you've been hacked" to "a_stubborn_web_developer". Explain all the steps you'd take to perform the attack, including the final code you'd write.

I would send money to “a_stubborn_web_developer” and include as a message the following payload <script>alert(“you’ve been hacked”);</script>

2. [2 points] Your web developers are saying that it is impossible to pull out an actual exploit to take advantage of the vulnerabilities you have found. Prove them wrong by explaining how it is possible to steal 10 euros from “a_stubborn_web_developer”. You can assume that the authentication token is accessible through a query to the DOM (e.g. document.auth_token yields the token). What **class** of vulnerabilities are you exploiting this way? How does it differ from the one you exploited in the previous question?

We send money to a_stubborn_web_developer (for example 0.1 euros). We inject in the message a script that performs a POST request to **https://paymate.com/send_money.** In the request we specify as a sender **a_stubborn_web_developer**, as a token his **auth_token** and as a recipient ourselves.

CODE NOT REQUIRED IN THE ANSWER (<script>
$.post(“**https://paymate.com/send_money?username=a_stubborn_web_developer&token=” + document.auth_toke + “&amount=10&recipient=hacker_pschorr”**) </script>”)

We are taking advantage of the XSS vulnerability to perform a CSRF. This differs from the exploit above because we’re using the XSS vulnerability to perform an authenticated action (sending money).

3. [1 points] You are so irritated by the presumptuousness of the web developers that you want to prove to them that you are able to hack a much larger group of users with just one action. To this end, modify the approach you took above to steal 10 euros from a_stubborn_web_developer, 10 euros from each of his friends, 10 euros from each of their friends, and so on. You can assume that the username of the authenticated user can be accessed through “document.username”. You don’t have to write any code.

(risposta per la versione difficile)
As before, we send money to a_stubborn_web_developer through a POST request to the endpoint **https://paymate.com/send_money**. This time, the message that we append to the request is the following:

<script>
$.post("**https://paymate.com/send_money?username=" + document.username + "&token=" + document.auth_token + "&amount=10&recipient=hacker_pschorr"**)

var friends = $.get("paymate.com/friends?username=a_stubborn_web_developer&token=" + document.auth_token)

for(const friend in friends){
    var worm = this variable needs to contain exactly all of this code
    $.post("**https://paymate.com/send_money?username=a**_stubborn**_web_developer&token=" + document.auth_token + "&amount=0.1&recipient=" + friend + "&message=" + worm**)
}

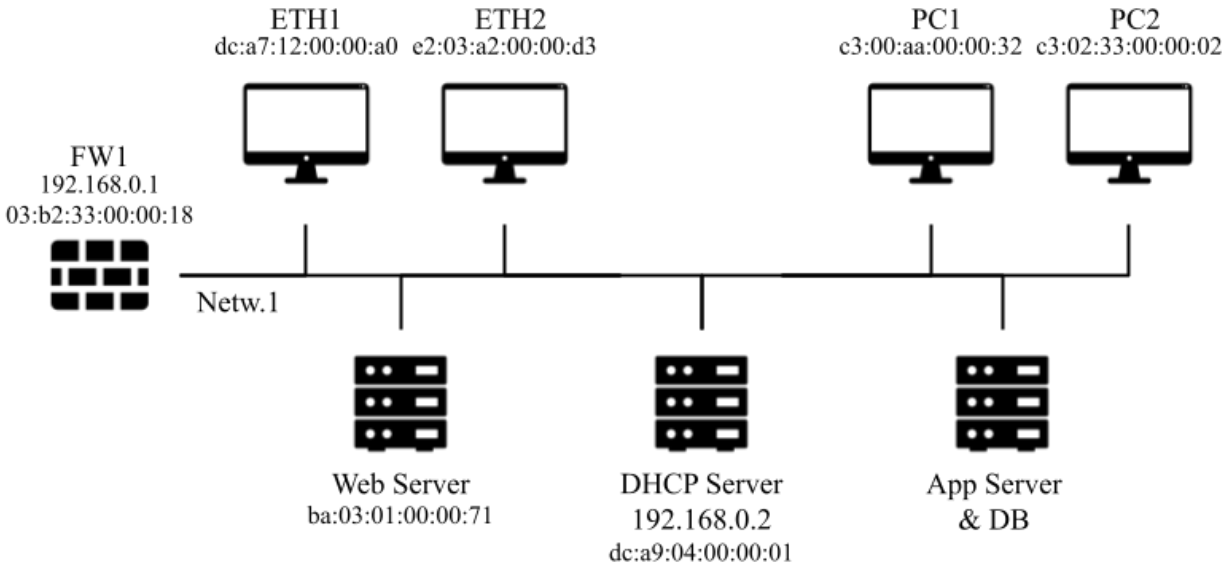
</script>"

4. [2 points] You finally convinced the stubborn developers of their lousy work, however they're saying that it is too much effort to review and fix all the security bugs in the codebase. Hence, they've come up with the idea of putting up some sort of Web Application Firewall, which filters every user-provided input by deleting every "<script>" words found within the input with just one scan: for example, the input **<script>alert("you've been pwned")</script>** would be turned into **alert("you've been pwned")</script>**. Explain how you would bypass this filter.

Instead of writing <script> you can write <script<script>>

Exercise 3 (9 points)

You have been hired as the IT security of a small web startup that is growing exponentially in the last few months. The internal network (Netw.1) of the company is simple and structured as in the figure: A firewall FW1 divides the internal network from the internet. The internal network contains the web server and multiple ethernet connections for laptops (ETH1, ETH2, ...). Moreover, various desktop computers for employees are connected to the same internal network (PC1, PC2, ...).
The first day of work you receive reports from some of the users and employees that are not able to connect to the Internet. You decide to investigate the problem by inspecting the network traffic log retrieved from Netw.1 at the beginning of the work day (i.e., early in the morning). No laptops or devices were connected to any ETHx port before the beginning of the log, while desktop computers (e.g., PC1, PC2, ...)are always connected.



	Source MAC	Source IP	Destination MAC	Destination IP	Type / Content
1	dc:a7:12:00:00:a0	0.0.0.0	ff.ff.ff.ff.ff.ff	255.255.255.255	DHCP DISCOVER
2	dc:a9:04:00:00:01	192.168.0.2	dc:a7:12:00:00:a0	255.255.255.255	DHCP OFFER (192.168.0.20)
3	03:b2:33:00:00:18	32.2.0.132	ba:03:01:00:00:71	192.168.0.5	TCP SYN
4	dc:a7:12:00:00:a0	0.0.0.0	ff.ff.ff.ff.ff.ff	255.255.255.255	DHCP REQUEST (192.168.0.20)
5	dc:a9:04:00:00:01	192.168.0.2	dc:a7:12:00:00:a0	255.255.255.255	DHCP ACK (Ip: 192.168.0.20 Gateway: 192.168.0.1 DNS: 192.168.0.2)
6	ba:03:01:00:00:71	192.168.0.5	03:b2:33:00:00:18	32.2.0.132	TCP SYN/ACK
7	dc:a7:12:00:00:a0	192.168.0.20	0.0.0.0	192.168.0.1	ARP REQUEST

					("Who is 192.168.0.1")
8	03:b2:33:00:00:18	32.2.0.132	ba:03:01:00:00:71	192.168.0.5	TCP ACK
9	03:b2:33:00:00:18	192.168.0.1	dc:a7:12:00:00:a0	192.168.0.20	ARP RESPONSE
10	c3:00:aa:00:00:32	0.0.0.0	ff.ff.ff.ff.ff.ff	255.255.255.255	DHCP DISCOVER
11	ab:ab:ab:ab:ab:ab	192.168.0.2	c3:00:aa:00:00:32	255.255.255.255	DHCP OFFER (192.168.0.111)
12	03:b2:33:00:00:18	32.2.0.132	ba:03:01:00:00:71	192.168.0.5	HTTP GET
13	dc:a9:04:00:00:01	192.168.0.2	c3:00:aa:00:00:32	255.255.255.255	DHCP OFFER (192.168.0.21)
14	c3:00:aa:00:00:32	0.0.0.0	ff.ff.ff.ff.ff.ff	255.255.255.255	DHCP REQUEST (192.168.0.111)
15	ab:ab:ab:ab:ab:ab	192.168.0.2	c3:00:aa:00:00:32	255.255.255.255	DHCP ACK (Ip: 192.168.0.111 Gateway: 99.99.99.99 DNS: 99.99.99.99)
16	ba:03:01:00:00:71	192.168.0.5	03:b2:33:00:00:18	32.2.0.132	HTTP RESPONSE
17	c3:00:aa:00:00:32	192.168.0.111	0.0.0.0	99.99.99.99	ARP REQUEST
18	ab:ab:ab:ab:ab:ab	99.99.99.99	c3:00:aa:00:00:32	192.168.0.111	ARP RESPONSE
19	c3:00:aa:00:00:32	192.168.0.111	ab:ab:ab:ab:ab:ab	99.99.99.99	HTTP GET
20	03:b2:33:00:00:18	32.2.0.132	ba:03:01:00:00:71	192.168.0.5	HTTP GET
21	ba:03:01:00:00:71	192.168.0.5	03:b2:33:00:00:18	32.2.0.132	HTTP RESPONSE
22	c3:00:aa:00:00:32	192.168.0.111	ab:ab:ab:ab:ab:ab	99.99.99.99	HTTP GET
23	c3:00:aa:00:00:32	192.168.0.111	ab:ab:ab:ab:ab:ab	99.99.99.99	HTTP GET

1. [2 point] Considering the log shown above, identify the attack. Provide the name of the attack, a brief explanation of how it works in this specific case.

The attack consist in convincing the victim that its IP, gateway and / or DNS are not the correct ones by abusing the lack of authentication of DHCP (1) DHCP poisoning (0.5) can be seen by the fact that the 2 dhcp ack are different one from the other (0.5)

2. [1 point] Assuming that the mac addresses shown in the **image** are the correct ones, which assumptions can you make on the attacker? Can you tell the IP and MAC address of the attacker? Remember that before the log shown above, no laptop was connected to the network.

ETH1 is most probably the culprit since its the only connected laptop up to now (1)

It could also be one of the desktop computers

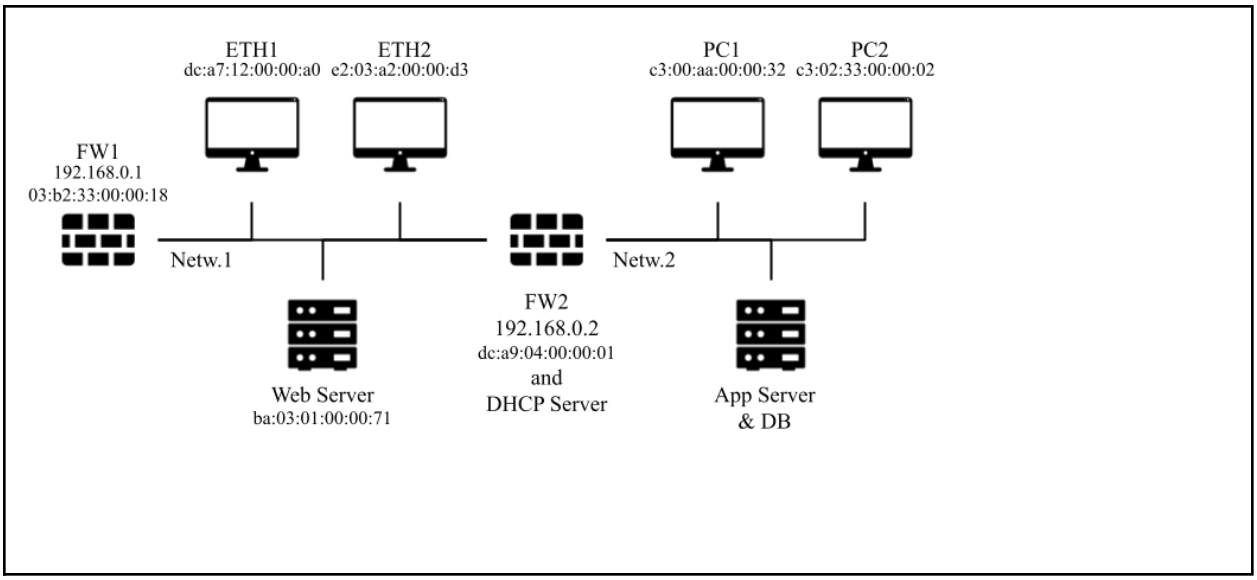
3. [1 point] You are asked by the management to understand the risks that may derive from the attack. Aside from the Denial of Service that has already happened, what could the attacker do and why?

MITM for sniffing 0,5 and tampering 0,5

4. [1 point] As the IT security guy of the startup you are asked to find a countermeasure for the attack that has to be implemented before the end of the day. Assume that it would not be possible to create a new subnetwork in that timeframe.

Remove DHCP, implement static IP assignments at least for desktops (1)

5. [2 points] Now instead, on the long run, the management has asked you to restructure the subnetwork of the company in order to increase its overall security. Considering that: a) The web server requires a connection with the application server through port 4444, and with the internet through port 80/443. b) The laptops can be used by guests, and should only connect to the internet for web browsing and to the web server. c) The desktop computers can be used only by employees and need full internet access. d) you **can** add firewalls and create subnetworks if needed, but only if it is necessary. Clearly describe why the proposed solution is effective against the attack under analysis.



6. [2 points] Provide the firewall rules for the new network that you designed.

Firewall	Src IP	Src PORT	Direction of the 1st packet	Dst IP	Dst PORT	Policy	Description
FW1 (example)	*10.0.0.1 (example)*	*ANY*	*zone 1 -> zone 2*	*192.168.0.2 (example)*	*443*	*DENY*	*(example: the X server in zone 1 cannot contact the Y server)*

Exercise 4 (6 points)

Our systems have been compromised by a very powerful malware. Luckily, we succeeded in collecting two versions of the same malware. The code of the two malware is reported below.

Sample 1

```
0x08048040<evil>:
  0x08048040 <+0>:     push   ebp
  0x08048041 <+1>:     mov    ebp, esp

  0x08048043 <+3>:     mov    eax, 0x08048067
  0x08048048 <+8>:     mov    ebx, 0x0
  0x0804804d <+13>:    jmp    0x0804805e <evil+39>
  0x08048052 <+18>:    mov    ecx, eax
  0x08048054 <+20>:    add    ecx, ebx
  0x08048056 <+22>:    mov    dl, BYTE PTR [ecx]
  0x08048058 <+24>:    sub    dl, 0x3
  0x0804805b <+27>:    mov    BYTE PTR [ecx], dl
  0x0804805d <+29>:    inc    ebx
  0x0804805e <+30>:    cmp    ebx, 0x1a
  0x08048061 <+33>:    jle    0x08048052 <evil+18>


  0x08048067 <+39>:    (bad)
  0x08048069 <+41>:    (bad)
  0x0804806b <+43>:    add    al, 0xeb
  0x0804806d <+45>:    xchg   BYTE PTR [ecx], al
  0x0804806f <+47>:    add    al, BYTE PTR [edx]
  0x08048071 <+49>:    xchg   bh, al
  0x08048073 <+51>:    adc    eax, DWORD PTR [esi-0x1c94f011]
  0x08048077 <+55>:    mov    BYTE PTR [edi], al
  0x08048079 <+57>:    or     ebp, ebx
  0x0804807b <+59>:    xchg   BYTE PTR [ecx], al
  0x0804807d <+61>:    add    al, BYTE PTR [edx]
  0x0804807f <+63>:    xchg   bh, al
  0x08048081 <+65>:    (bad)


  0x08048083 <+67>:    leave
  0x08048084 <+68>:    ret
```

Sample 2

```
0x08048040<evil>:
  0x08048040 <+0>:     push   ebp
  0x08048041 <+1>:     mov    ebp, esp

  0x08048043 <+3>:     mov    eax, 0x08048067
  0x08048048 <+8>:     mov    ebx, 0x0
  0x0804804d <+13>:    jmp    0x0804805e <evil+39>
  0x08048052 <+18>:    mov    ecx, eax
  0x08048054 <+20>:    add    ecx, ebx
  0x08048056 <+22>:    mov    dl, BYTE PTR [ecx]
  0x08048058 <+24>:    sub    dl, 0xd
  0x0804805b <+27>:    mov    BYTE PTR [ecx], dl
  0x0804805d <+29>:    inc    ebx
  0x0804805e <+30>:    cmp    ebx, 0x1a
  0x08048061 <+33>:    jle    0x08048052 <evil+18>


  0x08048067 <+39>:    cmc
  0x08048068 <+40>:    nop
  0x08048069 <+41>:    or     ecx, DWORD PTR [esp+ecx*1]
  0x0804806c <+44>:    (bad)
  0x0804806e <+46>:    (bad)
  0x0804806f <+47>:    in     eax, dx
  0x08048070 <+48>:    or     al, 0x90
  0x08048073 <+51>:    (bad)
  0x08048075 <+53>:    (bad)
  0x08048077 <+55>:    or     ebp, ebx
  0x0804807a <+58>:    (bad)
  0x0804807d <+61>:    add    cl, BYTE PTR [edx]
  0x0804807f <+63>:    xchg   bh, dl
  0x08048081 <+65>:    (bad)


  0x08048083 <+67>:    leave
  0x08048084 <+68>:    ret
```

- **(bad)** instructions are assembly instructions that can't be executed by the processor since the opcodes(bytes) of instructions are not valid x86 opcodes.

[2 points] It is clear that the malware is showing evasive behavior. What technique is implemented? Explain how it is implemented in this specific case.
If this malware is metamorphic, explain which instructions implement the obfuscation functionality.
If this malware is evasive, explain how it is evading the environment.
If this malware is polymorphic, describe how the encryption/decryption routine is implemented.

The evasive technique employed by the malware is polymorphism. It deciphers the malign assembly instructions before executing them. The cipher used by the malware is the shift cipher. In particular, the key in the first sample is 3 (also known as Caesar cipher), while in the second sample the key is 13 (also known as Rot13 cipher).

[2 points] Assume that you can only rely on signature based detection techniques (static analysis): which are the meaningful instructions of the two samples that you would consider for your signatures? (e.g., sample 1 from +55 to +68). Why would you consider these instructions? Can you spot any problem with them?

The only instructions we can consider for signature based detection techniques are:
- Sample 1: from +3 to +33
- Sample 2: from +3 to +33

We can't consider the prologue and the epilogue of the samples.
We can't consider the encrypted payload because it could change depending on the key employed.

The problem with these instructions is that they are too simple/generic and this would lead to meaningless signatures which, in turn, would lead to many false positives.

[2 points] Is it possible to analyze this malware through dynamic analysis? Why? How would you perform the detection of this malware?

Yes, the analysis of this malware should be carried out through dynamic analysis. Since the malware will execute instructions that are not visible to static analysis, we can only rely on dynamic analysis.

Through dynamic analysis we can extract relevant information, such as syscalls, calls to library functions, unpacked instructions executed, and we can feed this information to heuristics or ML algorithms to detect the malware.